

Plugin API Guide

YaPcore supports **three** extension kinds: Folia-aware / Spigot-style plugins, YaP plugins, and **modules**. See [MODULES_AND_API.md](#) for modules + coverage. Compatibility matrix: [PLUGIN_COMPAT.md](#).

Product game: YaP-Folia (`game-authority=folia` , `folia-jar-source=build`) — not stock Folia, not Paper. Build with `./scripts/build-yap-folia.sh`.

One folder: drop first-party Folia-native jars into `plugins/`. `folia-kernel/plugins` (and legacy `paper-kernel/plugins`) symlink to that folder.

First-party plugins use `YapSched` (Folia `GlobalRegionScheduler` / entity / region affinity) and declare `folia-supported: true` . Stock Paper jars are **unsupported** on the product path.

`yap-spatial-tick.jar` is **not** a product plugin — Paper Phase 3 legacy only (benches). YaP-Folia refuses it (`folia-supported: false`).

```
cp MyPlugin.jar plugins/
./scripts/start.sh --fg
```

1. Legacy Spigot / Paper / Purpur (`plugin.yml`)

On YaP-Folia (default — production path)

Put first-party jars in `plugins/`. They must declare `folia-supported: true` and schedule via `com.yapcore.sched.YapSched` (or Folia region APIs directly).

On Paper (legacy benches)

Put jars in `plugins/`. Within reason, anything that works on stock Paper 26.2 works here when `game-authority=paper` . Folia-only plugins may not.

YaP Compatibility Bridge (non-game authority only)

When Paper/YaP-Folia is not game authority, lightweight `plugin.yml` jars can load through the Compatibility Bridge (soft-fail). Prefer Folia authority for production plugins.

Bukkit call (facade)	YaPcore routing
<code>Bukkit.getScheduler().runTask(...)</code>	Compatibility Bridge → GameCore (SYNC)
<code>runTaskAsynchronously(...)</code>	Heavy I/O pool

Bukkit call (facade)	YaPcore routing
inventory / block helpers	Bridged → SYNC where implemented

2. Next-gen YaP plugins (yap.yml) — same plugins/ folder

New plugins extend `com.yapcore.api.YaPPlugin` and declare work on the right pool:

```
getScheduler().runHeavy(() -> {
    boolean ok = database.charge(player, price);
    if (ok) {
        getScheduler().runSync(() -> {
            player.getInventory().addItem(reward);
            getScheduler().runUi(() -> openSuccessMenu(player));
        });
    }
});
```

yap.yml

```
name: MegaStore
main: com.example.MegaStorePlugin
version: 1.0.0.0
api: yap-1
author: You
description: All-in-one store (GUI + economy + DB) without lag
```

3. Modules (modules/ + module.yml)

Same pools as YaP plugins; intended for **optional fine-tuning** operators add like mods. See [MODULES_AND_API.md](#).

4. Vehicles (plugin + API)

Real vehicle mechanics for plugin authors (cars / bikes / custom chassis — **not** minecarts or boats): [VEHICLES.md](#). Soft-depend `YaPVehicles` and load `VehicleAPI` from `ServicesManager`. Runs on **YaP-Folia** (`folia-supported: true`).

Shipped jars

CORE + NETWORK (default) on `gradle shadowJar` / `assembleRelease` :

yap-placeholderapi, yap-pregen, yap-plugin-compat, yap-db, yap-perms, yap-playerdata, yap-moderation, yap-essentials, yap-admin, yap-packs, yap-chat, yap-tab, yap-discord, yap-protect, yap-world, yap-regions, yap-guard, yap-lagguard, yap-map, yap-npcs, yap-factions, yap-floodgate, yap-bedrock-ui, yap-folia-bridge.

GAMEPLAY opt-in (`gradle installGameplayDefaults` or `-PyapGameplay=true`): yap-vehicles, yap-stacker, yap-gameplay-knobs, plus MMO:

Jar	Plugin
yap-skills.jar	YaPSkills — 13 RS skills
yap-combat.jar	YaPCombat — custom combat
yap-crafting.jar	YaPCrafting — recipes
yap-mmo-content.jar	YaPMmoContent — quests/bosses
yap-abilities.jar	YaPAilities — combat abilities
yap-mmo-bedrock.jar	YaPMmoBedrock — Bedrock forms
yap-mechanics.jar	YaPMechanics — stamina/nodes
yap-games.jar	YaPGames — minigames
yap-guilds.jar	YaPGuilds — guilds

SQL plugin authors: `compileOnly(project(":yap-db-api"))` and soft-depend `YaPDB` ([YAPDB.md](#)). Ranks: [PERMISSIONS.md](#).

Crash reports

On faults, watchdog recoveries, or `crashdump` console command, YaPcore writes:

`logs/crashes/crash-<timestamp>-<kind>.log`

Including: full thread dumps, heap, JVM/OS, config, loaded plugins/modules, bridge queue stats, metrics, and recent console output.